

The Module Dataset and How to Use It

Environment, data dictionary, the cyberlab.py program, and the public datasets worth knowing

Module	CITY2161 Data Modelling and Machine Learning for Cyber Security (FHEQ Level 5)
For	Every workshop, Topics 1–10, and the practical assessment
Tools	Python 3.12+; NumPy, pandas, SciPy, scikit-learn, imbalanced-learn, matplotlib, shap

Every workshop in this module works on the same dataset, and every number in the lecture notes was produced from it by programs you can run. That is deliberate: the module is one pipeline built over ten weeks, and a result you can reproduce is a result you can question. This kit gives you the environment, the program that generates the data, a description of every column, and an honest account of how the dataset differs from a real network and from the public benchmarks.

1. Setting up once

Create one virtual environment for the whole module and install the pinned versions below. Newer versions will run the workshops; the numbers in the notes may then differ in the last digit.

```
1 python3 -m venv .venv
2 source .venv/bin/activate
3 pip install numpy==2.5.3
4 pip install pandas==3.0.5
5 pip install scipy==1.18.1
6 pip install scikit-learn==1.9.1
7 pip install imbalanced-learn==0.14.2
8 pip install matplotlib==3.11.2
9 pip install shap==0.52.0
```

Listing 1: the module environment (Linux/macOS shell; on Windows use .venv\Scripts\activate).

Save cyberlab.py (Listing 2) in the folder where your notebooks live. Every workshop then begins with the same two lines.

```
1 import cyberlab as C
2 X = C.load()           # 7,200 host-hours: features, label and metadata
```

Listing 3: the first two lines of every workshop.

2. What the dataset simulates

cyberlab.py simulates a small enterprise network: 60 hosts observed for 120 hours, producing 131,903 network flows. In each hour a host behaves normally or performs one of three attacks — a port scan, data exfiltration or SSH brute force — with attacks becoming slightly more frequent later in the period. Normal behaviour includes two deliberate look-alikes: nightly backups, which move large volumes over long sessions like exfiltration, and busy administrators, who touch many ports like a scanner. An alerting system raises alerts on most attack hours and on a few benign ones.

Two views of the data are available. C.make_flows() returns the raw flows and alerts, for the SQL work of Topic 2. C.host_hour_matrix() — or simply C.load() — aggregates them to one row per host per hour, which every other topic uses.

3. Data dictionary

Table 1: the host-hour matrix. C.NUM and C.CAT list the seven feature columns.

Column	Role	Meaning
host_id, ts	Keys	The host and the hour the row describes
n_flows	Feature (numeric)	Flows the host started in the hour
n_distinct_ports	Feature (numeric)	Distinct destination ports contacted
n_distinct_dsts	Feature (numeric)	Distinct destination addresses contacted
total_bytes	Feature (numeric)	Bytes sent across all flows
mean_duration	Feature (numeric)	Mean flow duration in seconds
failed_logins	Feature (numeric)	Failed login attempts
top_service	Feature (categorical)	The most frequent service: http, dns, ssh, smtp or other
label	Target	1 if the alerting system raised a true-attack alert in the hour
n_alerts	Metadata — LEAKS	Alerts raised in the hour; derived from the label's source, never a feature (Topic 2)
attack_family	Metadata	portscan, exfil, bruteforce or benign, from the alert; for grading and held-out-family tests
true_kind	Audit only	What the host really did, with C.load(return_truth=True); no real network has this column

TWO COLUMNS THAT MUST NEVER BE FEATURES

n_alerts is the label in disguise; using it gives a perfect score and a detector that detects nothing (Topic 2). true_kind is the answer key the simulation keeps so that label quality can be audited (Topics 4 and 10); a real network never has it.

4. Dataset card

Table 2: what is true of this dataset, and where in the module each property is used.

Property	Value	Why it matters
Host-hours	7,200	Small enough to run anything on a laptop in seconds
Labelled attack rate	6.6%	Hundreds of times richer in attacks than a real network — precision here will not transfer (Topics 1, 9)
Families (labelled)	250 scans, 112 exfiltration, 110 brute force	Held-out-family experiments (Topic 7)
Real attack hours	567	95 (17%) never alerted, so labelled benign (Topics 4, 10)
False-positive alerts	154	Benign hours on which the alerting system fired
Attack rate by period	5.4% / 7.8% / 8.9%	First 60% / next 20% / last 20% of hours; the world moves under a model (Topic 9)

Table 2: what is true of this dataset, and where in the module each property is used.

Property	Value	Why it matters
Total bytes, median / 99th percentile	42,824 / 3,463,089	Heavy tails: summarise with medians, transform with log1p (Topics 1, 3)
Row independence	Each hour generated independently	A random split leaks less here than on real traffic, where campaigns persist — still split by time (Topic 9)

5. The program

```

1  """cyberlab.py -- one synthetic security dataset for the whole of CITY2161.
2
3  Every workshop, every staff-guide program and every number quoted in the
4  lecture notes is computed from this file, so a learner can reproduce any
5  figure in the notes by editing a program they already have open.
6
7  Why synthetic? A public benchmark cannot give two things this module needs:
8  it is not reproducible to the digit across library versions, and nobody knows
9  its ground truth. Here we do. `make_flows(return_truth=True)` also returns
10 what really happened in each host-hour, which is how Topic 10 measures label
11 noise: the labelling detector misses about 15% of attack hours, exactly as a
12 real alert-derived label would.
13
14 Views
15 ----
16 make_flows()          raw flows + alerts          (Topic 2 SQL work)
17 host_hour_matrix()    one row per host-hour        (Topics 1, 3-10)
18
19 Columns in the host-hour matrix
20 -----
21 FEATURES  n_flows, n_distinct_ports, n_distinct_dsts, total_bytes,
22            mean_duration, failed_logins          (numeric)
23            top_service                            (categorical)
24 LABEL     label          1 if a TRUE alert fired in the host-hour
25 METADATA  n_alerts       alert count -- derived from the label's source, so
26            attack_family portscan / exfil / bruteforce / benign, read from the
27            alert signature -- metadata for held-out-family tests,
28            never a feature
29            true_kind      (only when truth is passed) what the host really did,
30            alerted or not -- for measuring label noise
31
32 """
33 import numpy as np
34 import pandas as pd
35
36 NUM = ["n_flows", "n_distinct_ports", "n_distinct_dsts", "total_bytes",
37        "mean_duration", "failed_logins"]
38 CAT = ["top_service"]
39 FAMILY = {1: "portscan", 2: "exfil", 3: "bruteforce"}
40

```

```

41
42 def make_flows(seed=42, n_hosts=60, n_hours=120, attack_frac=0.08, return_truth=False):
43     """Return (flows, alerts) DataFrames, or (flows, alerts, truth) when
44     return_truth=True. One latent behaviour per host-hour drives the traffic;
45     attacks are slightly more frequent later (a trend Topic 9 exploits for the
46     temporal split and drift work)."""
47     rng = np.random.default_rng(seed)
48     hours = pd.date_range("2024-03-01", periods=n_hours, freq="h")
49     flow_rows, alert_rows, truth_rows, fid = [], [], [], 0
50     for host in range(n_hosts):
51         host_id = f"h{host:03d}"
52         for hi, ts in enumerate(hours):
53             p = attack_frac * (0.6 + 0.8 * hi / n_hours) # rising trend
54             kind = "benign"
55             if rng.random() < p:
56                 kind = rng.choice(["portscan", "exfil", "bruteforce"],
57                                   p=[0.5, 0.25, 0.25])
58             if kind == "benign":
59                 quirk = rng.random()
60                 if quirk < 0.04: # nightly backup: looks like exfil
61                     n = 1 + rng.poisson(4)
62                     ports = rng.choice([443, 22], size=n)
63                     dsts = rng.integers(0, 2, size=n)
64                     bytes_ = rng.lognormal(12.5, 0.9, size=n).astype(int)
65                     dur = rng.lognormal(2.5, 0.8, size=n)
66                     fail = np.zeros(n, int)
67                 elif quirk < 0.08: # busy admin: looks like a scan
68                     n = 20 + rng.poisson(30)
69                     ports = rng.integers(1, 400, size=n)
70                     dsts = rng.integers(0, 40, size=n)
71                     bytes_ = rng.lognormal(7.5, 1.0, size=n).astype(int)
72                     dur = rng.lognormal(-1.0, 1.0, size=n)
73                     fail = np.zeros(n, int)
74                 else: # ordinary traffic
75                     n = 1 + rng.poisson(rng.gamma(2.0, 6.0))
76                     ports = rng.choice([80, 443, 22, 53, 25], size=n,
77                                         p=[0.4, 0.3, 0.08, 0.15, 0.07])
78                     dsts = rng.integers(0, 8, size=n)
79                     bytes_ = rng.lognormal(7.5, 1.3, size=n).astype(int)
80                     dur = rng.lognormal(-1.0, 1.0, size=n)
81                     fail = (rng.random(n) < 0.01).astype(int)
82             elif kind == "portscan":
83                 inten = rng.uniform(0.3, 1.0) # some scans stealthy
84                 n = 10 + rng.poisson(int(150 * inten))
85                 ports = rng.integers(1, int(64 + 960 * inten), size=n)
86                 dsts = rng.integers(0, int(10 + 190 * inten), size=n)
87                 bytes_ = rng.lognormal(4.0, 0.6, size=n).astype(int)
88                 dur = rng.lognormal(-3.0, 0.5, size=n)
89                 fail = np.zeros(n, int)
90             elif kind == "exfil":
91                 inten = rng.uniform(0.4, 1.0) # low-and-slow vs bulk
92                 n = 1 + rng.poisson(3)
93                 ports = rng.choice([443, 22], size=n)
94                 dsts = np.zeros(n, int)

```

```

95         bytes_ = rng.lognormal(11.0 + 3.0 * inten, 0.8, size=n).astype(int)
96         dur = rng.lognormal(3.0, 0.8, size=n)
97         fail = np.zeros(n, int)
98     else: # bruteforce
99         inten = rng.uniform(0.3, 1.0)
100         n = 8 + rng.poisson(int(70 * inten))
101         ports = np.full(n, 22)
102         dsts = rng.integers(0, 3, size=n)
103         bytes_ = rng.lognormal(5.0, 0.5, size=n).astype(int)
104         dur = rng.lognormal(-1.5, 0.5, size=n)
105         fail = (rng.random(n) < 0.6 + 0.35 * inten).astype(int)
106     svc = {80: "http", 443: "http", 53: "dns", 22: "ssh", 25: "smtp"}
107     for j in range(n):
108         port = int(ports[j])
109         proto = "udp" if port == 53 else "tcp"
110         flow_rows.append((fid, host_id, ts,
111             f"10.0.{dsts[j] // 256}.{dsts[j] % 256}", port, proto,
112             svc.get(port, "other"), int(bytes_[j]), float(dur[j]),
113             int(fail[j])))
114         fid += 1
115     fired = kind != "benign" and rng.random() < 0.85
116     if kind == "benign" and rng.random() < 0.02: # false-positive alert
117         fired = True
118     if fired:
119         sig = {"portscan": 1, "exfil": 2, "bruteforce": 3, "benign": 4}[kind]
120         alert_rows.append((len(alert_rows), host_id, ts, sig,
121             "high" if kind != "benign" else "low",
122             1 if kind != "benign" else 0))
123         truth_rows.append((host_id, ts, str(kind)))
124     flows = pd.DataFrame(flow_rows, columns=["flow_id", "host_id", "ts",
125         "dst_ip", "dst_port", "protocol", "service", "bytes", "duration",
126         "is_failed_login"])
127     alerts = pd.DataFrame(alert_rows, columns=["alert_id", "host_id", "ts",
128         "signature_id", "severity", "is_true_attack"])
129     if return_truth:
130         truth = pd.DataFrame(truth_rows, columns=["host_id", "ts", "true_kind"])
131     return flows, alerts, truth
132 return flows, alerts
133
134
135 def host_hour_matrix(flows, alerts, truth=None):
136     """Aggregate flows to one row per (host_id, hour) with a binary label."""
137     g = flows.groupby(["host_id", "ts"])
138     X = g.agg(n_flows=("flow_id", "size"),
139         n_distinct_ports=("dst_port", "nunique"),
140         n_distinct_dsts=("dst_ip", "nunique"),
141         total_bytes=("bytes", "sum"),
142         mean_duration=("duration", "mean"),
143         failed_logins=("is_failed_login", "sum")).reset_index()
144     top_svc = g["service"].agg(lambda s: s.value_counts().index[0]).rename("top_service")
145     X = X.merge(top_svc.reset_index(), on=["host_id", "ts"])
146     # n_alerts is derived from the label's source -> it LEAKS. Kept out of the
147     # feature list on purpose; Topics 2 and 9 use it to demonstrate leakage.
148     ac = alerts.groupby(["host_id", "ts"]).size().rename("n_alerts").reset_index()

```

```

149     X = X.merge(ac, on=["host_id", "ts"], how="left")
150     X["n_alerts"] = X["n_alerts"].fillna(0).astype(int)
151     true_alerts = alerts[alerts.is_true_attack == 1]
152     fam = (true_alerts.groupby(["host_id", "ts"])["signature_id"].first()
153           .map(FAMILY).rename("attack_family").reset_index())
154     X = X.merge(fam, on=["host_id", "ts"], how="left")
155     X["attack_family"] = X["attack_family"].fillna("benign")
156     X["label"] = (X["attack_family"] != "benign").astype(int)
157     if truth is not None:
158         X = X.merge(truth, on=["host_id", "ts"], how="left")
159     return X.sort_values(["ts", "host_id"], kind="stable").reset_index(drop=True)
160
161
162 def load(return_truth=False):
163     """The host-hour matrix in one call: X = cyberlab.load()."""
164     if return_truth:
165         flows, alerts, truth = make_flows(return_truth=True)
166         return host_hour_matrix(flows, alerts, truth)
167     return host_hour_matrix(*make_flows())

```

Listing 2: cyberlab.py — save this file beside your notebooks.

6. Public datasets, and what they allow

The practical assessment may use this dataset or a public one. Public intrusion datasets are valuable because they contain real traffic, but each has known defects. Read its known-issues literature before trusting any score on it, and state the version you used.

Table 3: public intrusion-detection datasets and their known issues. References are in the Topic 9 notes.

Dataset	What it is	Known issues	Temporal split possible?
KDD Cup 1999	Features from simulated 1998 military network traffic	About 78% of training and 75% of test records duplicated (Tavallae et al., 2009)	No — do not use for new work
NSL-KDD	KDD'99 with duplicates removed (Tavallae et al., 2009)	Still 1998 traffic; records carry no timestamps	No
UNSW-NB15	Modern synthetic attacks and real background traffic (Moustafa & Slay, 2015)	Check the released train/test partition's class balance against your deployment	Only where record times are retained in the release you use
CICIDS2017	Five days of traffic with labelled attacks and 80+ flow features (Sharafaldin et al., 2018)	Flow-construction and labelling errors (Engelen et al., 2021; Lanvin et al., 2022)	Yes — use the corrected release of Engelen et al. (2021)

7. Reproducing the numbers in the notes

Every measured figure in the slides, notes and staff guide is read at build time from JSON files written by three programs in the module's lab directory: `exp_foundations.py` (Topics 1–4), `exp_models.py` (Topics 5–8) and `exp_deploy.py` (Topics 9–10). The committed results were produced with Python 3.12.14, scikit-learn 1.9.1, pandas 3.0.5, NumPy 2.5.3. If a number in the notes disagrees with what your version of a workshop program prints, check the random seed, the split and the library versions first — and if it still disagrees, you may have found something worth reporting.